

CLINICAL PROGRAMMING BOOTCAMP • MODULE 03

Your Toolkit

SAS & R for clinical programming

Two languages, one job. Everything you build in this bootcamp, you'll build twice — and see how each language gets there.

No programming experience assumed.



Why you'll learn both

SAS

The established standard

- Decades of use in regulatory submissions
- Commercial, licensed, formally validated
- Still the default at many sponsors and CROs
- Submission datasets are shipped as SAS transport files

R

The fast-growing challenger

- Free and open source
- Huge package ecosystem for analysis and graphics
- Increasingly accepted by regulators
- Growing use for exploration, figures, and Shiny apps

Most teams today use both. Being fluent in either makes you far more employable — and helps you check your own work.

Every data task is the same seven moves



Learn these seven once. Then you only have to learn how each language spells them.

Building an SDTM domain is these moves, combined and repeated.

A SAS program is DATA steps and PROC steps

DATA step

Creates or changes a dataset. Reads one row at a time, applies your logic, writes rows out. This is where variables are built.

PROC step

Calls a ready-made procedure to do something with a dataset: print it, sort it, count it, summarize it.

```
data vs_clean;                                /* DATA step: build a dataset */
  set work.vs;                                /* read every row of WORK.VS  */
  if sysbp > 140 then flag = "HIGH";
run;

proc freq data = vs_clean;                    /* PROC step: count it      */
  tables flag;
run;
```

Every statement ends with a semicolon ; and every step ends with run;

An R script is a pipeline of verbs

Functions & verbs

Each action is a function: `filter()`, `select()`, `mutate()`. They take a table and return a new table.

The pipe `|>`

Sends the result on the left into the function on the right. Read it as the word “then”.

```
vs_clean <- vs |>                                # take vs, THEN
  mutate(
    flag = if_else(SYSBP > 140, "HIGH", NA_character_)
  )

vs_clean |> count(flag)                          # then count it
```

`<-` assigns a result to a name. `#` starts a comment. No semicolons needed.

Read a CSV file

SAS

```
proc import
  datafile = "dm_raw.csv"
  out  = work.dm
  dbms = csv replace;
  getnames = yes;
run;
```

R

```
dm <- read_csv(
  "dm_raw.csv",
  col_types = cols(
    SITEID = col_character()
  )
)
```

Both read the same file. R lets you declare column types inline; in SAS you often fix types afterwards.

Watch out: an ID like 001 looks numeric. If either tool guesses “number”, you lose the leading zeros and get 1. Always check your types after reading — PROC CONTENTS in SAS, glimpse() in R.

Look at what you have

SAS

```
/* structure: types & lengths */  
proc contents data = work.dm;  
run;  
  
/* the rows */  
proc print data = work.dm (obs=5);  
run;
```

R

```
# structure: types & sample values  
glimpse(dm)  
  
# the rows  
head(dm, 5)
```

Always look before you map. Half of all mapping bugs are visible in the first five rows.

?

How many rows and columns?

?

Which columns are text vs numbers vs dates?

?

Are there missing values?

?

Do the values look like what the CRF asked for?

Keep the rows and columns you want

SAS

```
/* rows */  
data drugA;  
    set dm;  
    where arm = "Drug A";  
run;
```

R

```
# rows  
drugA <- dm |>  
    filter(ARM == "Drug A")
```

SAS

```
/* columns */  
data small;  
    set dm;  
    keep usubjid age sex;  
run;
```

R

```
# columns  
small <- dm |>  
    select(USUBJID, AGE, SEX)
```

Rows: WHERE ↔ filter(). Columns: KEEP ↔ select(). Note R uses == to test equality, = to name arguments.

Create new variables (the heart of mapping)

SAS

```
data dm2;
  set dm;
  length usubjid $20 sex_c $1;
  usubjid = catx("-", studyid,
                put(siteid, z2.),
                put(subjid, z3.));
  if sex = 1 then sex_c = "M";
  else if sex = 2 then sex_c = "F";
run;
```

R

```
dm2 <- dm |>
  mutate(
    USUBJID = paste(STUDYID, SITEID,
                    SUBJID, sep = "-"),
    SEX_C = case_when(
      SEX == 1 ~ "M",
      SEX == 2 ~ "F",
      TRUE ~ NA_character_
    )
  )
```

This is exactly what SDTM mapping is — read raw columns, write standardized ones. IF/THEN/ELSE ↔ case_when() · catx() ↔ paste()

Why declare length in SAS? If you assign text without a LENGTH statement, SAS fixes the width from the first value it sees and silently truncates longer ones. R has no such trap — character vectors size themselves.

Sort, count, and summarize

SAS

```
proc sort data = dm out = dm_s;  
  by siteid subjid;  
run;
```

R

```
dm_s <- dm |>  
  arrange(SITEID, SUBJID)
```

SAS

```
/* counts */  
proc freq data = dm;  
  tables arm;  
run;  
  
/* numbers */  
proc means data = vs mean;  
  class visit;  
  var weight;  
run;
```

R

```
# counts  
dm |> count(ARM)  
  
# numbers  
vs |>  
  group_by(VISIT) |>  
  summarise(  
    mean_wt = mean(WEIGHT)  
  )
```

PROC FREQ is for categories; PROC MEANS is for numbers. In R, count() and group_by() + summarise() do the same jobs.

Combining datasets

SAS

```
/* must be sorted first! */  
proc sort data = dm;  by usubjid; run;  
proc sort data = ex;  by usubjid; run;  
  
data dm_ex;  
  merge dm (in = a) ex;  
  by usubjid;  
  if a;          /* keep DM rows */  
run;
```

R

```
# no sorting required  
dm_ex <- dm |>  
  left_join(ex, by = "USUBJID")
```

Same result, very different ceremony. SAS MERGE needs sorted inputs; R joins do not.

Joining is how a subject's data gets assembled

You'll join constantly: attaching the reference start date from EX onto VS to compute study day, or pulling treatment arm from DM onto every other domain. USUBJID is almost always the key.

The Rosetta Stone

You want to...	SAS	R (tidyverse)
Read a CSV	<code>PROC IMPORT</code>	<code>read_csv()</code>
See structure	<code>PROC CONTENTS</code>	<code>glimpse()</code>
See rows	<code>PROC PRINT</code>	<code>head()</code> / <code>print</code>
Keep rows	<code>WHERE</code> / <code>IF</code>	<code>filter()</code>
Keep columns	<code>KEEP</code> / <code>DROP</code>	<code>select()</code>
New variable	<code>DATA</code> step assignment	<code>mutate()</code>
Multi-way logic	<code>IF</code> / <code>THEN</code> / <code>ELSE</code>	<code>case_when()</code>
Sort	<code>PROC SORT</code>	<code>arrange()</code>
Count categories	<code>PROC FREQ</code>	<code>count()</code>
Summarize numbers	<code>PROC MEANS</code>	<code>group_by()</code> + <code>summarise()</code>
Join datasets	<code>MERGE ... BY</code>	<code>left_join()</code>
Stack datasets	<code>SET a b;</code>	<code>bind_rows()</code>

SAME IDEA, DIFFERENT NAMES

Where your data lives

Concept	SAS	R
A table of data	<code>dataset</code>	<code>data frame / tibble</code>
A place to store tables	<code>library (LIBNAME)</code>	<code>your R environment / files</code>
Temporary scratch space	<code>WORK library</code>	<code>objects in the session</code>
Add-on functionality	<code>SAS procedures (built in)</code>	<code>packages (install.packages)</code>
A missing number	<code>.</code> (a single dot)	<code>NA</code>
A missing text value	<code>"</code> (empty string)	<code>NA_character_</code>

Missing values are the #1 cross-language gotcha — never treat a missing value as zero or as blank text.

Cross-language gotchas



Case sensitivity

R is case-sensitive: AGE and age are different. SAS variable names are not.



Equality test

R uses == to compare, = to name arguments. SAS uses = for both.



Missing in maths

In R, mean(x) is NA if any value is missing — use na.rm = TRUE.



Sorting before merge

SAS MERGE requires sorted inputs. R joins do not.



Text length

SAS truncates text silently if LENGTH is too short. Declare it first.



Dates

Both store dates as numbers under the hood, but SDTM wants ISO 8601 text.

How you'll run each language

SAS

Where SAS Studio / SAS OnDemand for Academics, or an installed SAS session at your company.

You write a .sas program file

You get a LOG (messages and errors) and OUTPUT (results). Read the log first — always.

R

Where RStudio or a Jupyter notebook with the R kernel.

You write an .R script or an .ipynb notebook

You get results printed in the console or under each cell. Errors appear in red.

In this bootcamp: SAS exercises are .sas scripts with a written walkthrough; R exercises are Jupyter notebooks.

WHAT'S NEXT

Time to write some code

1 **Notebook 01 • SAS** SAS basics — libraries, DATA step, PROC PRINT/SORT/FREQ/MEANS

2 **Notebook 01 • R** R & tidyverse basics — the same seven moves

3 **Both use** the ABC-01 mock data you already have in /data

4 **Then Module 04** Reading raw clinical data properly — and your first domain

Do both. Even if you already know one language, writing the other version is the fastest way to really learn it.